AGILE PROCESS

A practical, beginner-friendly guide to how Agile teams plan, build, test, and improve software.

Agile in one sentence	Agile is an iterative way of delivering value in small increments, learning from feedback, and adapting as needs change.
Why teams use it	Earlier feedback, better visibility, faster risk discovery, and the ability to adjust priorities as requirements evolve.
What Agile is not	It is not the absence of planning or documentation. Agile teams plan continuously and create documentation that supports delivery.

1. What is Agile?

Agile is a mindset and a family of approaches for delivering products through short feedback loops. Instead of waiting until the entire product is finished, a team works on a small, valuable slice, validates it with stakeholders, and uses what it learns to guide the next slice.

The Agile Manifesto emphasizes four values:

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan.

The items on the right still have value; Agile places greater emphasis on the items on the left.

2. Core principles in everyday work

- Deliver useful software frequently instead of saving all value for a final release.
- Invite customer and stakeholder feedback throughout development.
- Keep business, development, and testing aligned on the same goal.
- Make quality everyone's responsibility and improve the way the team works regularly.

Remember: Agile is about feedback, collaboration, and adaptability—not simply working in sprints.

THE AGILE DELIVERY LIFECYCLE

Agile work is iterative. Teams repeatedly select work, clarify it, build and test it, review the outcome, and improve their process.

Step	What happens	Typical outputs
1. Product vision	Clarify the problem, users, desired outcomes, and product direction.	Vision, goals, success measures
2. Backlog creation	Break needs into epics, features, and user stories; order them by value and risk.	Prioritized product backlog
3. Refinement and planning	Discuss acceptance criteria, dependencies, estimates, capacity, and the sprint goal.	Ready stories, sprint plan
4. Design and development	Design, code, review, integrate, and test small increments collaboratively.	Integrated working increment
5. Verification	Run relevant automated and exploratory tests; address defects and assess risks.	Test evidence, defect reports
6. Review and release	Demonstrate completed work, gather feedback, and release when appropriate.	Feedback, releasable increment
7. Retrospective	Inspect teamwork and delivery; choose practical improvements to try next.	Improvement actions

A simple flow

Plan → **Build** → **Test** → **Review** → **Improve**

The loop repeats. Feedback may change the backlog, design, acceptance criteria, or release plan. Teams should deliver only work that meets their agreed quality and completion standards.

Common planning horizons

- **Product roadmap:** broad direction and outcomes over a longer period; it should be revisited as evidence changes.
- **Release planning:** coordinate what may be delivered and the risks or dependencies involved.
- **Sprint or iteration planning:** agree on a near-term goal and select work the team believes it can complete.

Agile does not require every organization to use fixed-length sprints. Scrum commonly uses sprints; Kanban focuses on managing flow and limiting work in progress.

SCRUM: ROLES, EVENTS, AND ARTIFACTS

Scrum is a lightweight framework that implements Agile ideas through defined accountabilities, events, and artifacts. The following is a practical overview.

Accountabilities

Accountability	Main responsibility
Product Owner	Maximizes product value and manages the Product Backlog, including ordering work and clarifying outcomes.
Scrum Master	Helps establish Scrum, improves team effectiveness, and helps address impediments.
Developers	Create a usable Increment each Sprint; plan and carry out the work, including the quality activities needed to meet the Definition of Done.

Scrum events

Event	Purpose	Typical questions
Sprint	A fixed-length container for achieving the Sprint Goal and producing an Increment.	What valuable outcome can we achieve?
Sprint Planning	Set the Sprint Goal and plan the work.	Why is this Sprint valuable? What can be done? How?
Daily Scrum	Inspect progress toward the Sprint Goal and adapt the plan.	What needs to change to meet the goal?
Sprint Review	Inspect the outcome with stakeholders and adapt future work.	What did we learn? What should happen next?
Sprint Retrospective	Plan ways to improve quality and effectiveness.	What helped, what hindered, and what will we try?

Artifacts and commitments

- **Product Backlog — Product Goal:** the ordered, evolving list of work supports the long-term product objective.
- **Sprint Backlog — Sprint Goal:** the selected work and plan for achieving the Sprint Goal.
- **Increment — Definition of Done:** the integrated usable result meets the quality standard agreed by the Scrum Team.

Important: Scrum does not prescribe a separate QA role. Testing and quality are collaborative responsibilities. Organizations may still assign specialist QA engineers to provide deep testing expertise.

QUALITY ENGINEERING IN AGILE

Quality should be built into every iteration rather than postponed to a separate testing phase at the end. QA engineers help the team prevent defects, evaluate risk, and provide fast, trustworthy feedback.

Where QA contributes

Stage	QA activities	Useful evidence
Refinement	Review user stories, identify edge cases, clarify testability, and improve acceptance criteria.	Clear acceptance criteria; risk notes
Planning	Estimate testing effort, identify dependencies and test data needs, and agree on quality risks.	Test approach; capacity and risk view
Development	Collaborate on unit, API, integration, UI, security, and accessibility testing as appropriate.	Automated checks; code review feedback
Continuous integration	Run fast automated suites, report failures quickly, and help triage flaky tests.	Build results; reliable test reports
Sprint validation	Perform exploratory and risk-based testing; verify acceptance criteria and regression impact.	Test evidence; prioritized defects
Review and improvement	Explain quality status, analyze escaped defects, and improve coverage or testability.	Quality trends; action items

Definition of Ready vs Definition of Done

Definition of Ready (team practice)	Definition of Done (Scrum commitment)
A checklist the team may use to decide whether a story is sufficiently understood to start—for example, clear value, acceptance criteria, and known dependencies.	A shared quality standard for an Increment—for example, code reviewed, required tests passed, acceptance criteria met, and no unresolved defects that violate the standard.

Practical quality metrics

- **Escaped defects:** defects found after release; analyze severity and recurring causes.
- **Automated test reliability:** distinguish real product failures from flaky tests and infrastructure issues.
- **Lead time and cycle time:** understand how long work takes to flow from request to delivery.
- **Change failure and recovery:** assess release risk and how quickly service is restored when changes cause problems.
- **Customer outcomes:** combine technical metrics with user impact, support trends, and feedback.

Avoid using a single metric—such as test case count or code coverage—as proof of quality. Metrics should inform decisions, not encourage teams to game the numbers.

EXAMPLE, BEST PRACTICES & QUICK REFERENCE

Example: building a login feature

Goal: allow registered users to sign in securely and reach their account.

Work item	Example
User story	As a registered user, I want to sign in with valid credentials so that I can access my account.
Acceptance criteria	Valid credentials allow access; invalid credentials show a safe error; required fields are validated; repeated failures follow the agreed security policy; the experience works on supported devices.
Build and test	Developers add the feature and unit tests; QA checks API behavior, UI validation, negative paths, session handling, accessibility, and relevant security risks.
Done means	Acceptance criteria pass, required automated checks pass, code is reviewed, changes are integrated, and the team's Definition of Done is met.
Review feedback	Stakeholders try the increment and identify usability gaps or new requirements; the Product Owner orders follow-up work in the backlog.

Agile best practices

- Keep work items small, understandable, testable, and tied to a user or business outcome.
- Prioritize by value, risk, dependencies, and learning—not by urgency alone.
- Integrate work frequently and automate repeatable checks where they provide fast, reliable feedback.
- Make work visible with a board and explicit policies; limit work in progress when using a flow-based method.
- Raise blockers and quality risks early; do not hide unfinished work to meet a sprint target.
- Use reviews to learn and retrospectives to make a small number of measurable improvements.

Common terms

Term	Meaning	Term	Meaning
Epic	Large body of work split into smaller items.	**User story**	A short description of user need and value.
Backlog	Ordered list of work to consider.	**Increment**	Integrated usable product result.
Velocity	A team-specific planning aid, not a productivity comparison.	**Retrospective**	Meeting to improve how the team works.

Key takeaway: Agile teams deliver value in small increments, inspect results, adapt plans, and build quality into the work every day.

Further reading: The Agile Manifesto (agilemanifesto.org) and the official Scrum Guide (scrumguides.org).